Data Science Lab

Name: Vishal Rajesh Mahajan Experiment No: 5

Class: BE INFT B-1 Roll No: 1

Setup and Import Libraries

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
```

Load CIFAR-10 Dataset

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
class_names = ["airplane","automobile","bird","cat","deer","dog","frog","horse","ship","truck"]
print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Training data shape: (50000, 32, 32, 3)
Test data shape: (10000, 32, 32, 3)
```

Visualize Sample Images

```
plt.figure(figsize=(10,2))
for i in range(10):
    plt.subplot(1,10,i+1)
    plt.xticks([]); plt.yticks([])
    plt.imshow(x_train[i])
    plt.xlabel(class_names[y_train[i][0]])
plt.show()
```



Build CNN Model

```
model = models.Sequential([
    # 1st Convolution Block
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    # 2nd Block
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    # 3rd Block
    layers.Conv2D(64, (3,3), activation='relu'),

    # Flatten + Fully Connected Layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

```
Total params: 122,570 (478.79 KB)
Trainable params: 122,570 (478.79 KB)
Non-trainable params: 0 (0.00 B)
```

Compile Model

```
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
```

Train Model

```
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
```

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 14s 6ms/step - accuracy: 0.3503 - loss: 1.7480 - val_accuracy: 0.5628 - val_loss: 1.2417
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5798 - loss: 1.1804 - val_accuracy: 0.6082 - val_loss: 1.1117
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6424 - loss: 1.0135 - val_accuracy: 0.6636 - val_loss: 0.9609
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.6850 - loss: 0.8968 - val_accuracy: 0.6867 - val_loss: 0.8961
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7147 - loss: 0.8182 - val_accuracy: 0.6818 - val_loss: 0.9094
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7358 - loss: 0.7598 - val_accuracy: 0.7056 - val_loss: 0.8592
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.7559 - loss: 0.7012 - val_accuracy: 0.6986 - val_loss: 0.8660
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.7710 - loss: 0.6622 - val_accuracy: 0.7131 - val_loss: 0.8420
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.7818 - loss: 0.6209 - val_accuracy: 0.7195 - val_loss: 0.8332
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7976 - loss: 0.5708 - val_accuracy: 0.7154 - val_loss: 0.8661
```

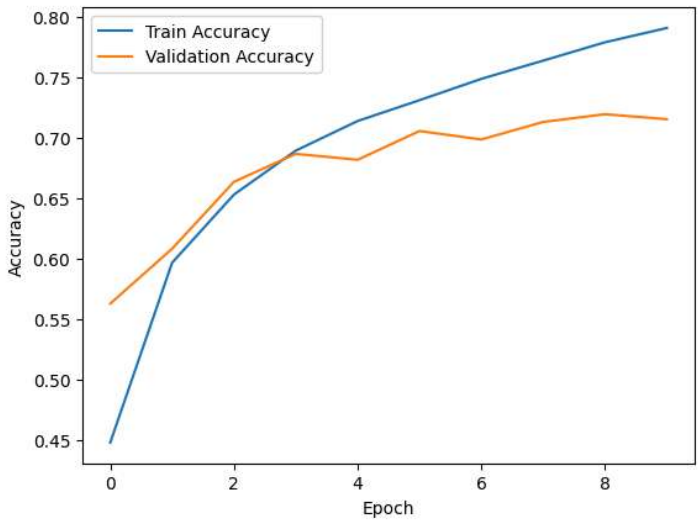
Evaluate on Test Data

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test accuracy: {test_acc:.2f}")
```

```
313/313 - 1s - 2ms/step - accuracy: 0.7154 - loss: 0.8661
Test accuracy: 0.72
```

Plot Training Curves

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



Make Predictions

```
idx = np.random.randint(0, x_test.shape[0], 10)
images, labels = x_test[idx], y_test[idx]

preds = model.predict(images)

plt.figure(figsize=(10,2))
for i in range(10):
    plt.subplot(1,10,i+1)
    plt.xticks([]); plt.yticks([])
    plt.imshow(images[i])
    pred_label = class_names[np.argmax(preds[i])]
    true_label = class_names[labels[i][0]]
    plt.xlabel(f"P:{pred_label}\nT:{true_label}")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 572ms/step
```

